KEEL CLIENT INTEGRATION MASTER PLAYBOOK

A Complete, Step-by-Step Guide for Client Meetings, Technical Setup, Agreements & Fee Collection

About This Guide: Written for non-technical operators and founders who need to meet with prospective clients (trading platforms, sportsbooks, iGaming, Web3 & Web2 apps), onboard them onto Keel, set up their credentials on your website, supervise developer testing, and collect agreed fee shares. No prior coding knowledge is assumed.

1. WHAT TO DO IN THE FIRST CLIENT MEETING (DISCOVERY & COMMERCIALS)

When meeting with a prospective client's CEO, Head of Product, or Engineering Lead, your goal is to understand their technical setup and establish commercial terms. Follow this exact agenda:

Step 1.1: Identify Their Technical Model (Ask This Exact Question)

Ask them: *'How does your platform hold and process player deposits today?'* They will fit into one of three models:

Model	How The Client Operates	What Keel Provides To Them
A. Web3 Smart Contract	Client deploys a vault or betting contract on EVM (Base, Ethereum, Polygon, Arbitrum).	Solidity interface (IKeelIntegrationV1.sol) + TypeScript SDK + REST API.
B. Web3 Server Custody	Client holds crypto via server wallets, MPC, or institutional custody (Fireblocks). No custom contract.	Direct server-to-server REST API + SDK. No contract deployment needed.
C. Web2 Platform	Traditional off-chain balances (fiat USD/EUR, gaming chips, loyalty points, sports bets).	Pure REST API + SDK tracking off-chain ledger units (zero blockchain overhead).

Step 1.2: Agree on Commercial Terms (How Fees Are Split)

- **Play Fee Share:** When an active player stakes protected balance, Keel charges an agreed percentage share (e.g. 5% of the play fee). You both agree on this percentage upfront.
- **Deposit Fee Share:** If the client turns on an optional deposit fee when users protect funds, Keel takes its agreed share of that fee too (including fees on auto-protected profits).
- **Exit Fee (100% to Client):** Any exit/withdrawal fees charged to the player belong entirely to the client. *Keel never takes any share of exit fees.*
- **Supported Asset:** Confirm which currency they use (e.g. USDC, USDT, ETH, SOL, or USD fiat).

2. ACTIONS ON THE KEEL OPERATOR SITE (YOUR DASHBOARD)

Once the meeting concludes and the client is ready to integrate, open your browser and go to **keelprotect.xyz/keel/owner**. Perform these 5 steps in order:

Step 2.1: Add Their Company in the Owner Console
Go to **Companies** in the left menu. Click **+ Add Company**. Enter their company name (e.g. 'Apex Sportsbook'). This provisions an isolated tenant space so their data and tests never touch any other partner.

Step 2.2: Enable Their Company Portal
Inside their company record, toggle **Portal Enabled** to ON. This unlocks their dedicated access to **keelprotect.xyz/keel/portal** so their engineers can view documentation, copy their API keys, and track their test progress without ever seeing your admin settings.

Step 2.3: Generate Their Sandbox Credentials
Under their Sandbox settings, click **Generate API Key** (starts with keel_sb_...). Then click **Generate Webhook Secret** (starts with whsec_...).
IMPORTANT SECURITY NOTICE: The webhook secret is shown *only once* upon creation. Copy it immediately into a password manager before closing the prompt!

Step 2.4: Draft and Sign the Billing Agreement
Navigate to **Billing** → **Agreements**. Click **New Agreement Draft**. Select the company, enter the Agreed Take Rate (e.g. 5%), select the billing token (e.g. USDC), and enter your designated fee payout wallet/address. Once finalized, click **Sign Agreement**.

Step 2.5: (Optional) Send an Integration Request Link
If you want their technical team to submit their token contract address or custody setup directly, generate a short link (e.g. keelprotect.xyz/i/<token>) from their company page and email it to them.

3. WHAT TO SEND TO THE CLIENT'S TEAM (WELCOME PACKAGE)

Copy and paste this message template to their engineering lead via email or Slack:

Client Onboarding Package Template:
• **Portal Login:** https://keelprotect.xyz/keel/portal (Login using the company username/password you provisioned).
• **Sandbox API Key:** keel_sb_...
• **Webhook Signing Secret:** whsec_...
• **Sandbox API Base URL:** https://keelprotect.xyz/api/public/keel/sandbox/v1
• **Public TypeScript SDK:** npm install @keelprotect/sdk
• **GitHub Repository:** https://github.com/lotechdgs/keel-sdk (Source code, types, & Solidity interface).
• **Direct Support:** hello@lotechdgs.com

4. WHAT THE CLIENT'S DEVELOPERS ACTUALLY DO (TECHNICAL STEPS)

If the client's management asks what their engineering team needs to build, explain these steps. A standard integration takes 1 to 2 developer days:

Step 4.1: Install the Toolkit
The developer runs one command in their server project:
npm install @keelprotect/sdk
(If they use Python, Go, or PHP, they can call the HTTPS REST API endpoints directly without the SDK).

Step 4.2: Import the Smart Contract (Web3 Contract Clients Only)
If they deploy an EVM contract (Base Sepolia, Ethereum, etc.), they import the Keel contract interface:
import "@keelprotect/sdk/contracts/IKeelIntegrationV1.sol";
This gives them standard functions: protectDeposit(), stakePlay(), closePlay(), exitProtected(), and claimKeelFees().

Step 4.3: Initialize the SDK in Their Backend Server
Their server initializes the Keel client with their Sandbox key:
import { KeelClient } from '@keelprotect/sdk';
const keel = new KeelClient({ apiKey: 'keel_sb_...', baseUrl: 'https://keelprotect.xyz/api/public/keel/sandbox/v1' });

Step 4.4: Connect the 4 Core Player Events
Their server calls Keel whenever a user takes action on their site:
1. **Deposit / Protect:** When a player deposits and opts into protection, call keel.protectDeposit().
2. **Stake / Wager:** When a player wagers from protected balance, call keel.stakePlay().
3. **Settlement & Auto-Profit:** When the game/trade ends, call keel.settlePlay(). If the player won, Keel automatically reserves 5–100% of their net profit according to their elected preference.
4. **Exit / Withdrawal:** When a player exits, call keel.exitProtected() (allowed only when protected reserve has not been fully used).

Step 4.5: Listen to Keel Webhooks
Keel sends instant webhook notifications whenever balances update or fees are charged. The client's backend verifies each message using the built-in helper:
keel.verifyWebhookSignature({ payload, signature, secret: process.env.KEEL_WEBHOOK_SECRET });

5. HOW FEE COLLECTION WORKS (HOW YOU GET PAID)

Here is the exact financial and mathematical mechanism of how fees are calculated, tracked, and transferred to your account:

5.1: The Fee Split Formula

Whenever a player stakes protected balance or pays a deposit fee, Keel calculates its agreed share down to the exact smallest currency unit (cent or wei) using the formula:

Keel Share Calculation:
keel_share = floor(eligible_fee_amount × agreed_take_rate)
Example: If a player incurs a $10.00 play fee, and Keel's agreed share is 5% (0.05):
• Keel's earned share = **$0.50**.
• Client retains = **$9.50**.
• *Important: Exit fees belong 100% to the client. Keel's share of exit fees is always $0.00.*

5.2: What You Must Give the Client for Settlement

Client Model	What You Provide to Client	How Settlement Occurs
Web3 Smart Contract	Your designated EVM/Base multi-sig or cold wallet address (0x...).	Their smart contract automatically routes accumulated Keel fees to your address, or you trigger claimKeelFees().
Web3 Custody / Server	Your crypto deposit address (USDC/USDT on Base, Ethereum, Solana, etc.).	Client transfers accumulated weekly/monthly statement balances from their custody wallet to your address.
Traditional Web2 / Fiat	Corporate bank wire / ACH / IBAN details and company tax identification.	Client pays monthly invoices generated directly from your Keel Owner Billing Console.

5.3: Automated Invoicing & Immutable Ledger

Inside your Keel Owner Console (**Billing** → **Ledger**), every single fee event is recorded with an exact-unit ledger entry. When generating invoices, Keel verifies accumulated uninvoiced earnings automatically, preventing any discrepancies, under-charging, or double-billing.

6. GOING LIVE — THE PRE-FLIGHT VERIFICATION GATES

Never issue Live API keys until both you and the client have passed these 6 verification gates:

[✓] Gate 1: Sandbox Checklist 100% Green: Inside their company portal, all flows (deposit, stake, settle, profit protection, exit, and webhooks) are successfully verified in TEST credits.

[✓] Gate 2: Bilateral Agreement Signed: The billing agreement in your owner dashboard is formally confirmed with agreed take rates and registered payout address.

[✓] Gate 3: Contract Bytecode Verified (Web3): For contract integrations, use Keel's built-in **Live Deployment Discovery** tool to confirm their deployed contract matches IKeelIntegrationV1.sol.

[✓] Gate 4: Generate Live Credentials: In the Owner Console, generate their Live API Key (keel_live_...) and Live Webhook Secret.

[✓] Gate 5: Client Switches Base URL: The client updates their backend environment configuration to: https://keelprotect.xyz/api/public/keel/live/v1

[✓] Gate 6: Live Smoke Test: Execute a small real-currency deposit and wager to verify webhook receipt, correct balance tracking, and ledger fee accrual.

7. SUMMARY INTEGRATION TIMELINE CHEAT SHEET

Timeline	Phase	Key Deliverable / Outcome
Day 1	Meeting & Commercials	Agree on model (Contract / Custody / Web2) & fee share (e.g. 5%).
Day 1	Keel Site Setup	Add Company in Owner Console, enable Portal, generate Sandbox API key & Webhook secret.
Day 2	Client Sandbox Integration	Client installs @keelprotect/sdk, connects testnet contract or server calls.
Day 3	Verification & Sign-off	Review Portal checklist, sign billing agreement, supply fee payout wallet.
Day 4+	Live Launch	Issue Live API key, switch endpoint to /api/public/keel/live/v1, start earning fees.